Pipelined RISC-V SoC

Summer 2026

Personal Project

By: Mekai Bingham

Table of Contents

Table of Figures

Table of Tables

1.0 Project Motivation

Many engineering projects rely on high-level abstractions that allow for rapid development, but come at the cost of a deep, technical understanding of the system. This has been my experience when working with hardware such as Arduino and Raspberry Pi, but also when writing in high-level languages such as Python. I wanted to work on a project where I could design a complete system from scratch. Inspired by my professor stating that designing a processor was possible with the material learned in my Computer Architecture course, I decided to try to build one myself and program a game on it.

2.0 Project Overview

This project implements a custom pipelined RISC-V SoC to run bare-metal Pong on a DE10Lite FPGA. Written in SystemVerilog, the SoC includes a CPU capable of running all 37/37 non-system instructions from the RV32I ISA, memory-mapped I/O, and VGA modules to interface with a display. The pipeline uses hazard detection and forwarding to resolve data and control hazards. The design meets timing on a 50MHz clock, with a slack of 0.648ns (Fmax = 51.67MHz), and a CPI of 1.343 when running Pong. Figure 1 shows the complete system, including the FPGA, a breadboard for the user interface, and the display. Figure 2 shows a GIF of the game in action, seen more clearly here.

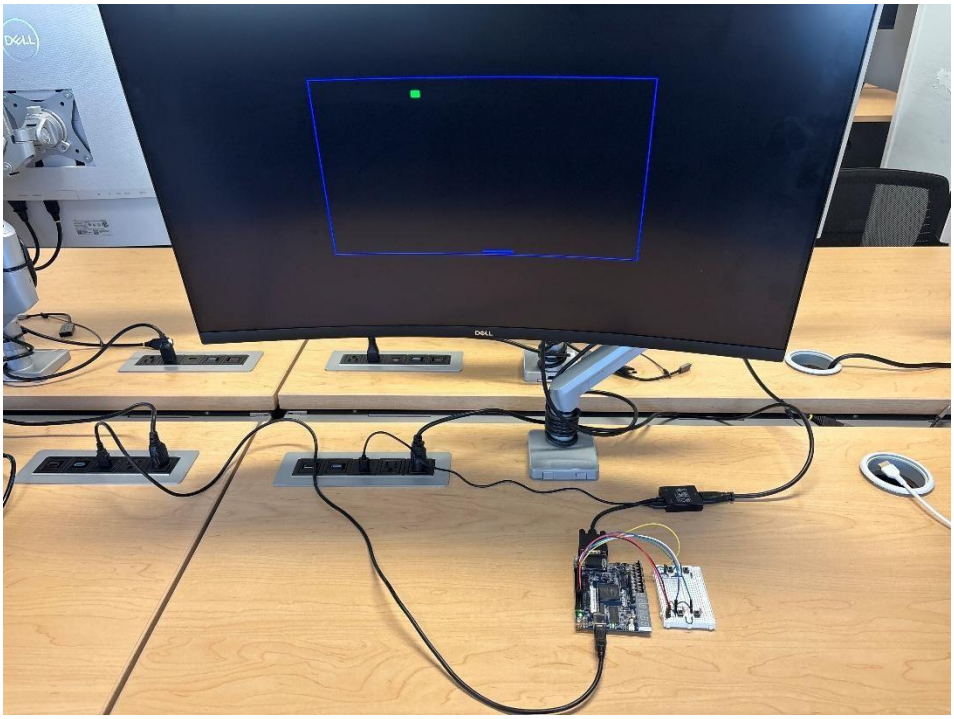

Figure 1: Picture of the complete system

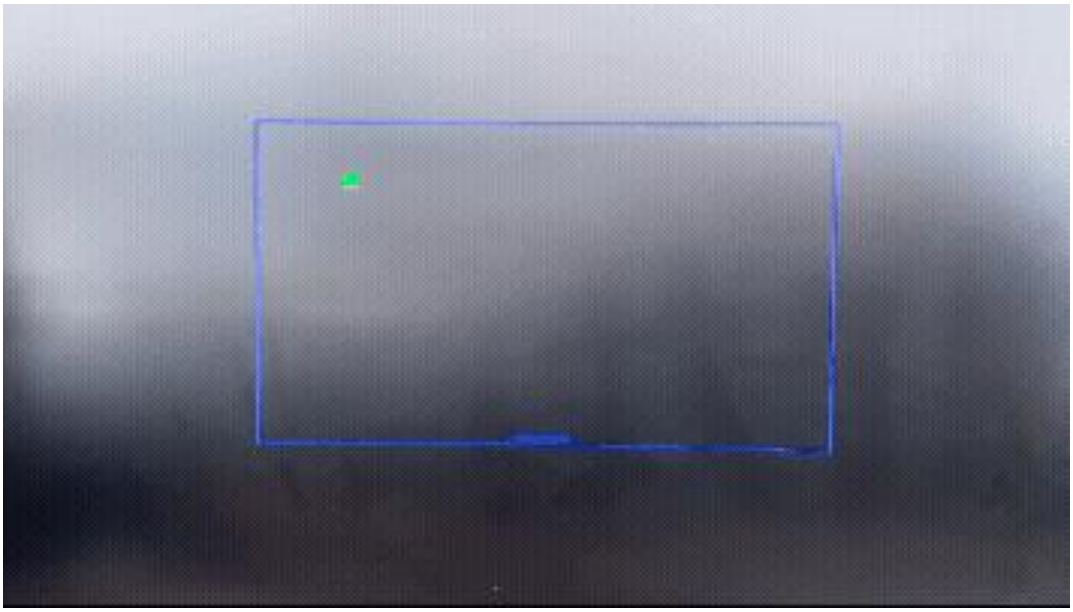

Figure 2: GIF showing Pong played on the RISC-V SoC

3.0 RISC-V Processor

The processor consists of many modules spread throughout the IF, ID, EX, MEM, and WB execution stages. All synchronous modules are run by a 50MHz clock on the FPGA. Figure 3 shows the high-level CPU datapath, which will be broken down into individual stages and modules in the following sections. The detailed datapath can be viewed in Figure 16. The datapath is based on the Patterson and Hennessy pipeline but expanded to include all RV32I non-system instructions.

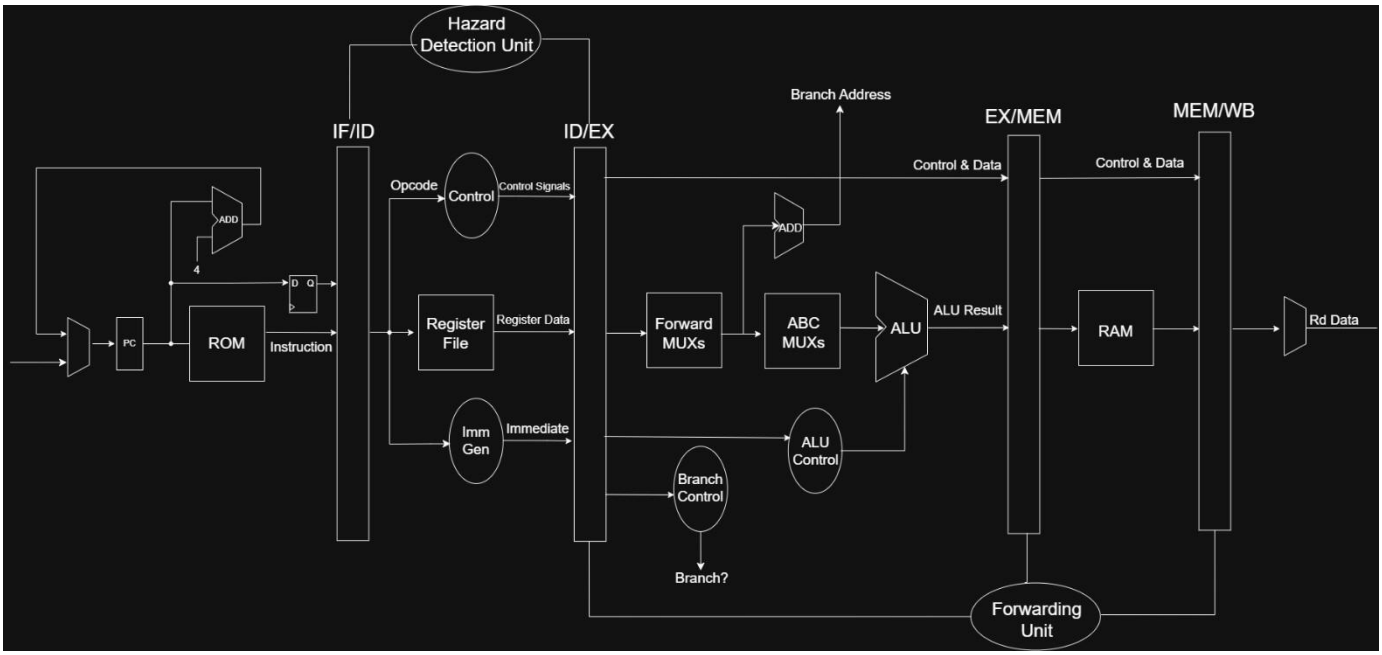

Figure 3: High Level RISC-V CPU Datapath

3.1 Instruction Fetch

The Instruction Fetch (IF) stage, seen in Figure 4, is responsible for incrementing the Program Counter (PC), and fetching the instruction at the corresponding address in the Instruction Memory (ROM).

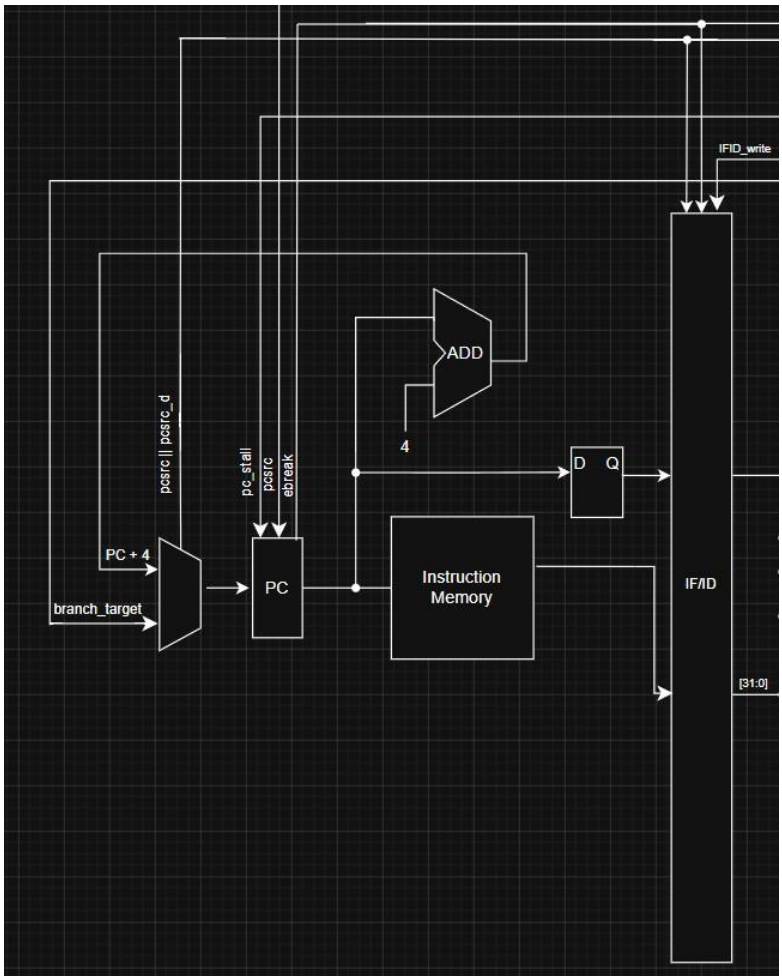

Figure 4: Instruction Fetch Stage

3.1.1 Program Counter

The PC provides the ROM with the memory address of the next instruction to be executed. The value of the PC is determined by an input mux which selects between a branch address (calculated in the EX-stage), or the current PC value + 4, depending on whether a branch was detected or not. The three control signals, pc_stall, pcsrc, ebreak, determine whether the PC should increment as normal, stall, or hold its value indefinitely. The PC stalls only when (pc_stall && !pcsrc) is true, i.e, when a hazard is signaled and no branch is taken. This was to prevent cases of both pcsrc and pc_stall being high on the same clock cycle, effectively killing the branch. The ebreak signal is triggered when the system instruction EBREAK is detected, causing the program counter to halt. The PC also takes clk and rst_n inputs not shown. Detailed explanations of the control signals can be found in Table 1 and Table 2.

3.1.2 Instruction Memory

The ROM holds the instructions to be executed. It occupies the first 5kB of BRAM, spanning the byte-addressable address range 0x0 to 0x13FF. It takes in the 32-bit address provided by the PC, and outputs the data corresponding to that address. Since the memory is 5kB and word-accessible, only 11 address bits are required, and the bottom two bits are ignored for word alignment. Hence bits [12:2] are used as the effective address. The control signal pc_stall is used to stall the ROM on a load/store hazard to ensure an instruction is not lost. It is important to note that the M9K memory blocks that make up the FPGA's BRAM do not support asynchronous reads, meaning the ROM and other memory modules are clocked. This introduced unexpected delays into the pipeline. This resulted in significant differences between what worked in simulation vs hardware.

3.1.3 IF/ID Buffer

The IF/ID Buffer is the first interstage buffer in the pipeline. It stores both the data and address of the instruction fetched from ROM. It takes three input signals which determine whether to stall or flush the buffer. IFID_stall stalls the buffer on a load-use hazard. The buffer is flushed twice on a branch by (pcsrc || pcsrc_d) to account for the stale instructions out of the ROM and PC on a control hazard. The buffer is also flushed on an ebreak signal to account for the bad instruction in ROM while an EBREAK instruction is preparing to halt the program.

3.2 Instruction Decode

The Instruction Decode stage (ID), seen in Figure 5, extracts the instruction's subfields (register addresses, immediate, and opcode), reads the source registers, and generates control signals.

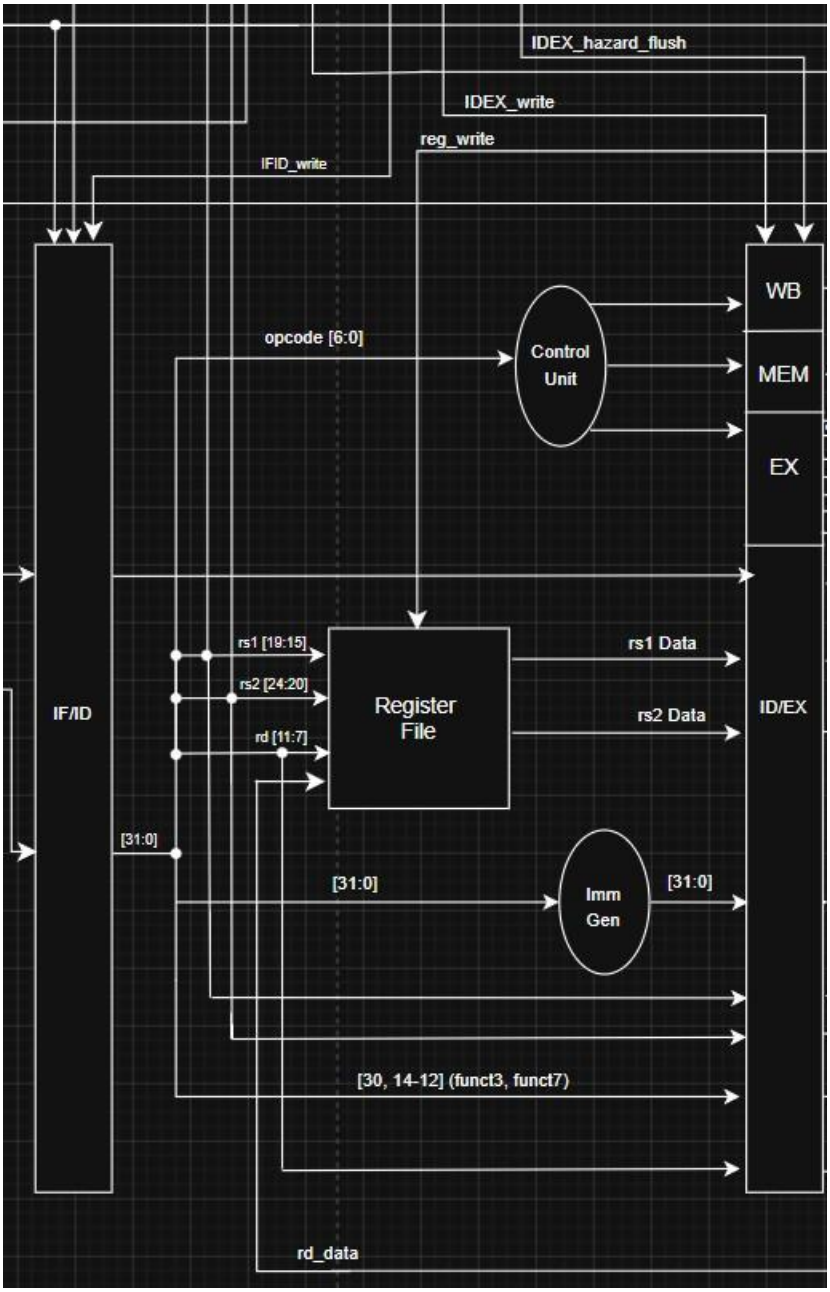

Figure 5: Instruction Decode Stage

3.2.1 Register File

The register file is a collection of 32, 32-bit registers. It receives the addresses of source register 1 and source register 2 from the IF/ID buffer directly, and receives the destination register address from the WB stage. Each register has an enable signal which allows it to be written to on a write and maintains its value otherwise. The register at address 0 (x0) cannot be written to, as it is hardwired to value 0.

3.2.2 Immediate Generation Unit

The Immediate Generation Unit is responsible for determining how to generate the appropriate immediate value depending on the specific instruction. Since each instruction type has a different binary format as seen in Figure 6, the unit must be able to recognize the type of instruction and then access different sections of said instruction to generate the complete immediate. It takes the full 32-bit instruction, recognizes the opcode, and then applies the appropriate combination of concatenation and sign-extension to generate a 32-bit immediate.

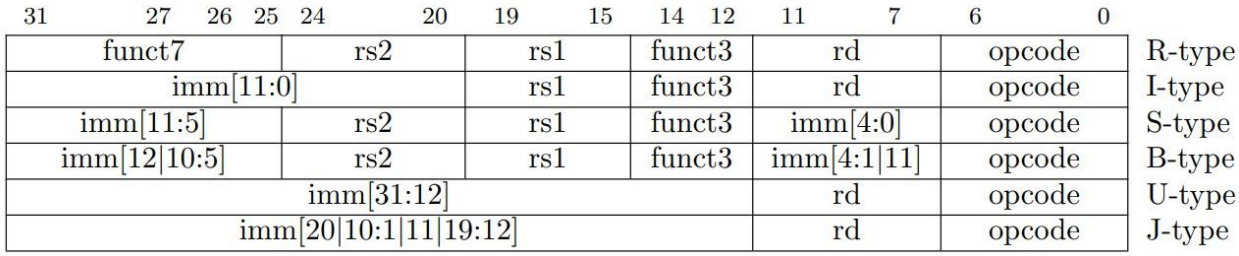

Figure 6: Binary encoding of different instruction types

3.2.3 Control Unit

The control unit decodes the opcode and generates control signals that determine how the remaining pipeline stages behave. These signals determine critical functions such as selecting the inputs to the ALU, deciding whether to read or write to memory, and more. A breakdown of the generated 1-bit and 2-bit control signals can be found in Table 1 and Table 2 respectively.

Table 1: 1-bit control signals

Signal	Stage Used	Purpose
btarget	EX	Selects between pc address and a forwarded result to be one operand for a branch calculation. Introduced to handle branch = pc + offset or branch = pc + rs1 data for J-type instructions
branch	EX	Allows the branch control unit to determine if a branch should be taken
jump	EX	Enables the branch control unit to branch on a J-type instruction
ebreak	EX	Allows the program to halt
mem_read	MEM	Issues a read command to RAM
mem_write	MEM	Issues a write command to RAM
reg_write	WB	Signals a write to a register in the register file
mem_to_reg	WB	Selects between memory data and the ALU result

Table 2: 2-bit control signals

Signal	Stage Used	Purpose
pc_to_alu	EX	Selects between a forwarded value (00), pc address (01), or constant 0 (10) as input 1 into the ALU
alusrc	EX	Selects between a forwarded value (00), immediate (01), or constant 4 (10) as input 2 into the ALU
aluop	EX	Supports funct3/funct7 in selecting the ALU operation

3.2.4 ID/EX Buffer

The ID/EX buffer is the second interstage buffer in the pipeline. It stores the control signals, register addresses, register data, and specific fields of the instruction. It takes two input signals that determine whether to stall or flush the buffer depending on the hazard type.

3.3 Execute

The Execute stage (EX), seen in Figure 7, is responsible for resolving the branch target and ALU result.

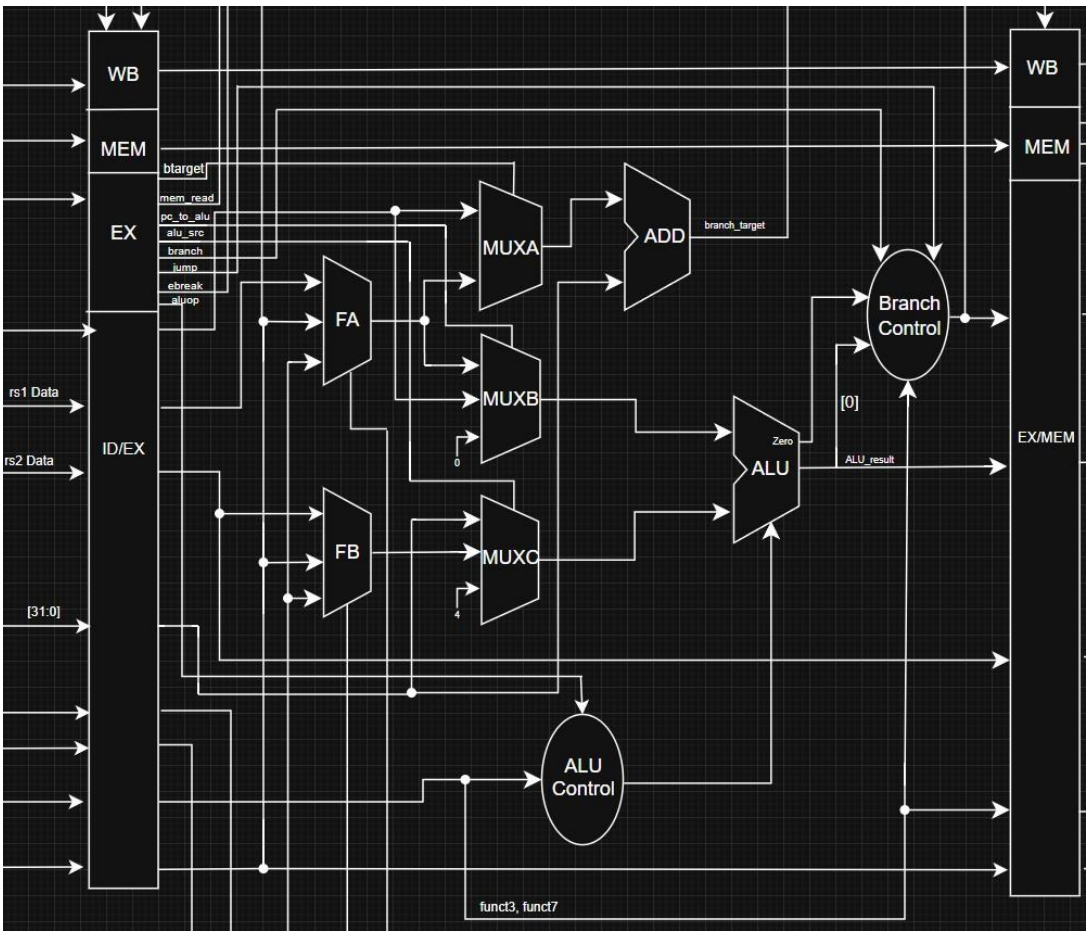

Figure 7: Execute Stage

3.3.1 MUX A

MUX A selects between the PC address or forwarded data, added to an immediate, to calculate the branch target. This MUX was introduced to account for J-type instructions. The Jump and Link Register (jalr) instruction requires that the adder must compute rs1 + offset, however, the Jump and Link (jal) instruction requires that the adder must compute pc + offset. MUX A was added to select between rs1 and the pc. Instead of directly using rs1 as an operand, it uses the forwarded value to account for hazards.

3.3.2 MUX B

MUX B selects between a forwarded value, the PC address, and a constant zero, as an input into the ALU. This MUX was also introduced to account for J-type instructions, since they require the ALU to compute PC + 4 as the return address. The zero input was added to allow the shifted 20-bit immediate (Shifted by Imm Gen) to pass right through the ALU on a Load Upper Immediate (LUI) instruction. This could have also been done by adding additional logic to tell the ALU to do nothing, but the current method reduces complexity.

3.3.3 MUX C

MUX C selects between a forwarded value, an immediate, and a constant 4. The forwarded and immediate inputs are selected during typical operation, and the 4 is selected on J-type instructions as mentioned earlier.

3.3.4 ALU Control Unit

The ALU control unit is responsible for determining the ALU operation based on funct3, funct7, and the aluop control signal. Aluop is used to tell the ALU control unit if it should perform an ADD, SUB, or if further inspection through funct3/funct7 is required to determine the operation. The unit uses this information to send a 4-bit control signal to the ALU.

3.3.5 ALU

The ALU takes two inputs and performs an arithmetic or logic operation determined by the ALU control unit. This includes addition, subtraction, bitwise AND/OR, and shifting. The ALU also generates a “zero” bit to tell the branch control unit whether the output was zero or not, which is used to determine if a branch should occur.

3.3.6 Branch Control Unit

The branch control unit is responsible for determining whether a branch should be taken or not. It uses the funct3 field to determine what signals it should be looking at, i.e, if funct3 corresponds to beq, check if the zero bit is high, or if it corresponds to blt, check the LSB of the ALU result. If these conditions are met, and the branch control signal is active, a branch is triggered and the output signal pcsrc goes high. The output signal goes high unconditionally whenever a jump instruction is detected.

3.3.7 EX/MEM Buffer

The EX/MEM buffer is the third interstage buffer in the pipeline. It holds the control signals for the MEM and WB stages, the ALU result, register addresses, register data, and instruction fields. It also holds the delayed pcsrc signal which accounts for an additional flush due to synchronous memory. It takes a signal, EXMEM_stall, which stalls the buffer on a hazard.

3.4 Memory

The Memory (MEM) stage, seen in Figure 8, is responsible for reading from or writing to data memory (RAM) using the address and data computed in the Execute stage.

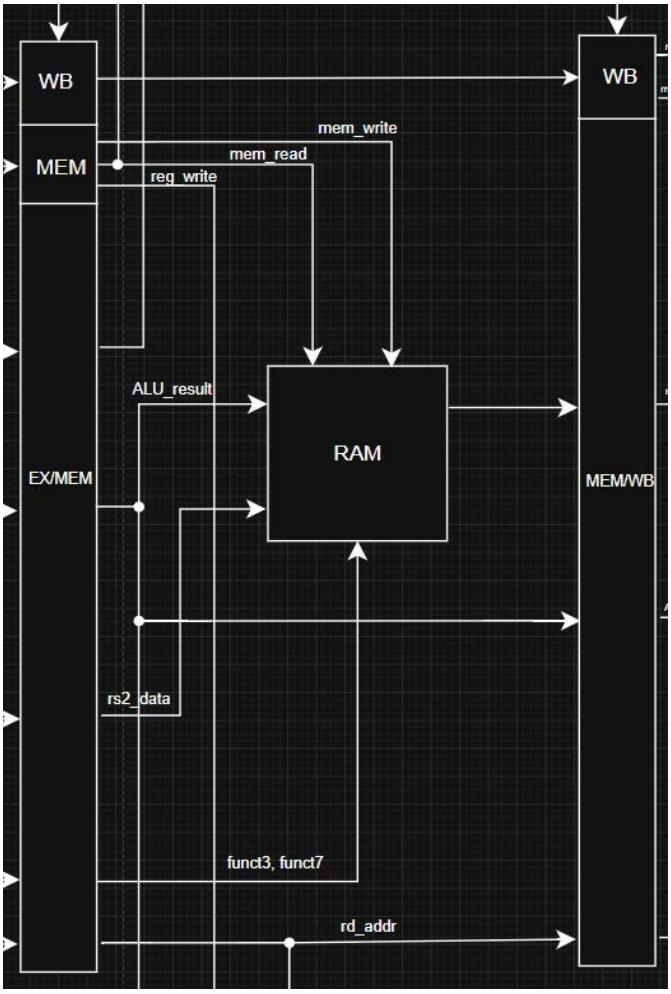

Figure 8: Memory Stage

3.4.1 RAM

The RAM holds the program data. It occupies range 5kB to 20kB in BRAM, corresponding to locations 0x1400 to 0x4FFF in memory. It takes the ALU result, used to determine the address to read from or write to, as well as data to be stored. The control signals mem_write and mem_read decide whether a read or write operation is performed. The RAM also takes the instruction's funct3 field to generate a byte-enable signal. This signal is used to allow memory operations (SB,

SH, LB, LH) to access an individual byte or halfword within a word, since the RAM array itself is only word-addressable. The lower two bits of the effective address select which byte or halfword within the word is targeted. Since the M9K BRAM does not support asynchronous reads, these selection signals must be registered so they remain valid one cycle later, once the word has actually been read out of memory. On a load, the registered read data is sliced and either sign-extended or zero-extended to 32 bits depending on the instruction. On a store, the same selection signals determine which bytes of the target word are overwritten with the incoming write data, leaving the remaining bytes in that word unchanged.

3.4.2 MEM/WB Buffer

The MEM/WB Buffer is the final interstage buffer in the pipeline. It holds the data read from RAM, the ALU result, destination register address, and control signals for the WB stage. It takes an input, MEMWB_hazard_flush, which flushes the buffer to prevent a double writeback on a hazard.

3.5 Writeback

The writeback stage, seen in Figure 9, is responsible for writing data back to the register file. The reg_write control signal is used to enable a specific register, and the mem_to_reg control signal determines the type of data to be written back. This data is either the ALU result, or the read data from RAM on a load instruction.

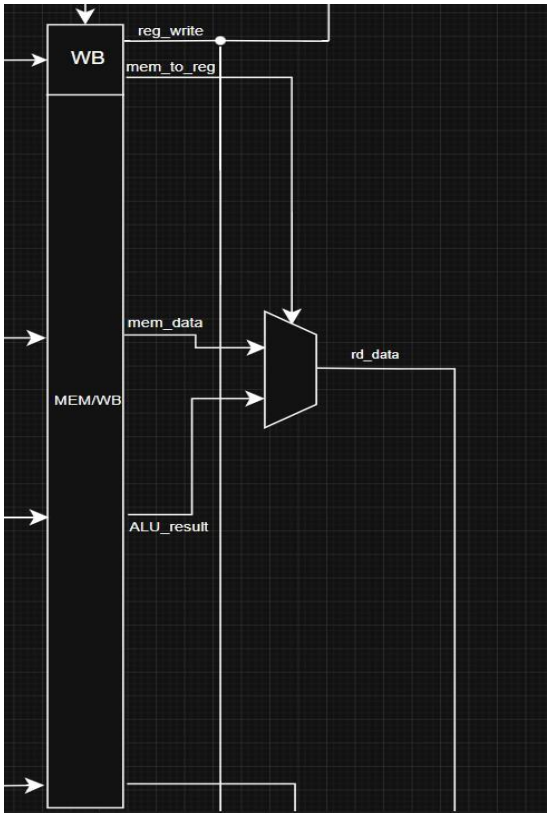

Figure 9: Writeback Stage

3.6 Hazard Detection

The pipeline uses a Hazard Detection Unit (HDU), seen in Figure 10, to handle data and control hazards. The unit is responsible for asserting the appropriate signals to flush or stall interstage buffers and the PC when a hazard is detected.

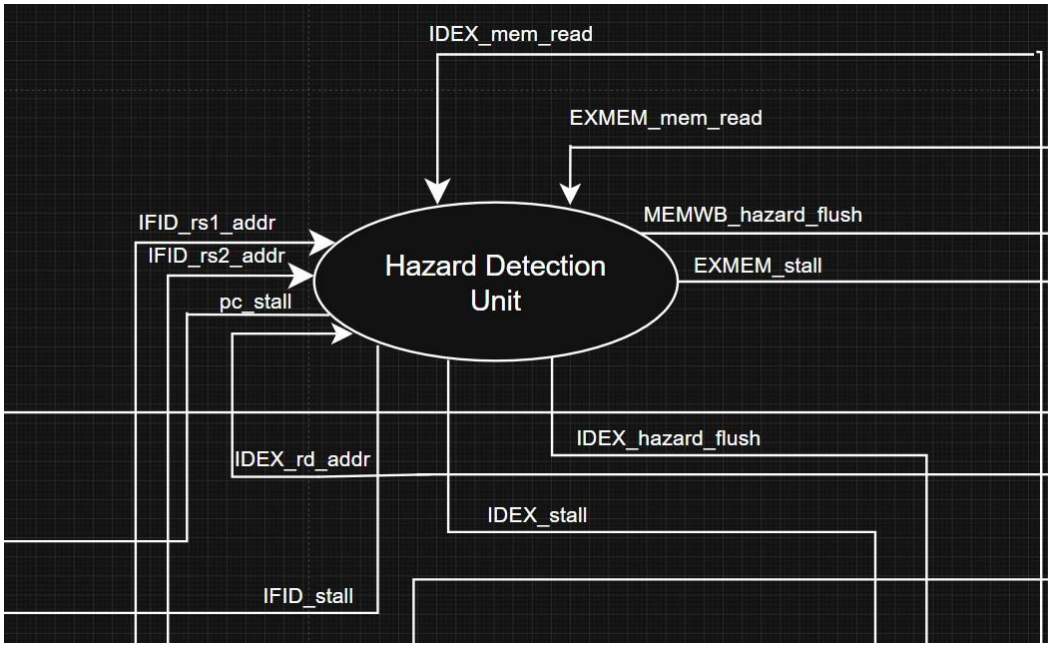

Figure 10: Hazard Detection Unit

3.6.1 Load-Use Hazards

A load-use hazard occurs when an instruction immediately following a load attempts to use the load's destination register as a source register. At the point the load completes and data is written to a register in the WB stage, the following instruction is already in the MEM stage, having used invalid register data. The solution to this is that when the load is in EX, insert a NOP into the EX stage on the next clock cycle while stalling all clocked modules earlier in the datapath. This creates an additional delay between the load and dependent instruction, so that the instructions are now two clock cycles apart. By the time the load is in WB, the dependent instruction is in EX instead of MEM, and data can be forwarded to EX via the forwarding unit. As an unfortunate consequence of synchronous memory reads, the pipeline downstream (towards IF stage) of the RAM must also be stalled for one clock cycle on any load instruction. The HDU handles this by unconditionally asserting stalls to clocked modules in the pipeline on a load.

3.6.2 Control Hazards

A control hazard occurs when a branch is asserted. Since a branch is executed in the EX stage, two undesired instructions are already in the IF/ID and ID/EX buffers, with a third being output from the synchronous ROM. This results in a branch delay of three instructions. To kill these instructions, the HDU inserts NOP's into the ID/EX and IF/ID buffers. The NOP is inserted twice into IF/ID to account for the instruction coming from ROM, since ROM doesn't have a flush signal.

3.7 Forwarding Unit

The Forwarding Unit, seen in Figure 11, allows data to be forwarded between different stages of the pipeline if required. For example, Section 3.6.1 above highlights the case where data must be forwarded from WB to EX for proper pipeline execution. The unit outputs control signals to the FA and FB MUXs, which are responsible for determining the inputs to MUX A, B and C in the EX stage. The Patterson and Hennessy textbook highlights two types of forwarding which the Forwarding Unit handles, as explained below.

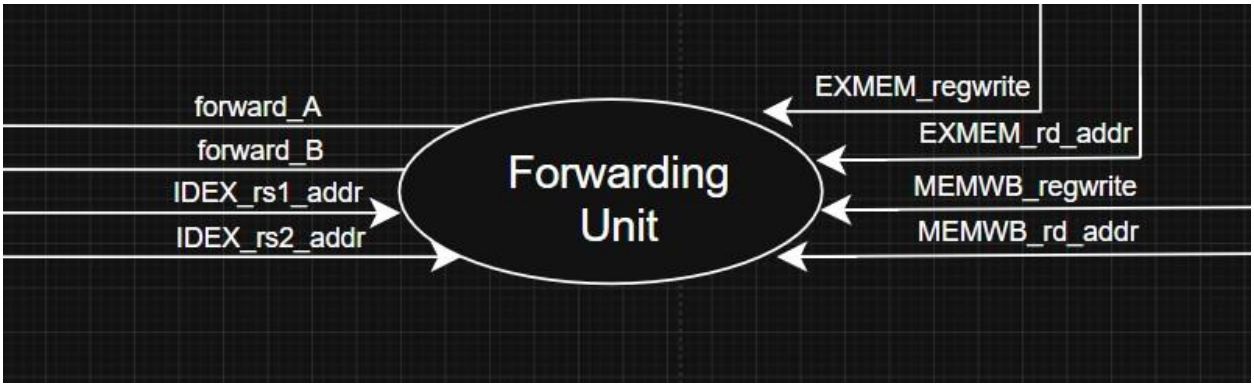

Figure 11: Forwarding Unit

3.7.1 WB to EX Forwarding

The first type of forwarding is when an instruction in EX requires data from the WB stage. This occurs primarily after a load-use hazard, when the dependent instruction needs a value from memory that has not been loaded yet. On the condition that the source register address of the dependent instruction in EX is equal to the destination register address of the initial instruction, the data is forwarded directly to MUXs in EX. This allows the dependant instruction to use this correct value rather than the invalid register data.

3.7.2 MEM to EX Forwarding

The second type of forwarding occurs when an instruction in EX requires data from the MEM stage. This arises when the dependent instruction needs the value of a register, not a value in memory, i.e, addi x2, x3, 0x1 , addi x2, x2, -0x2. When addi is in MEM, subi needs to use the value of x2 immediately during EX, and so rather than waiting for the data to be written to x2 during WB, the unit forwards (x3+0x1) directly to EX.

4.0 VGA Interface

To be able to display Pong on a monitor, a display interface was required. VGA was chosen due to its simplicity compared to digital communication protocols such as HDMI. A resolution of 640x480 pixels was used due to having strong documentation, as well as being able to fit well within the BRAM capacity of the FPGA. The implemented VGA interface consists of a Phase Locked-Loop (PLL) and multiple modules explained below. The diagram of the interface is shown in Figure 12.

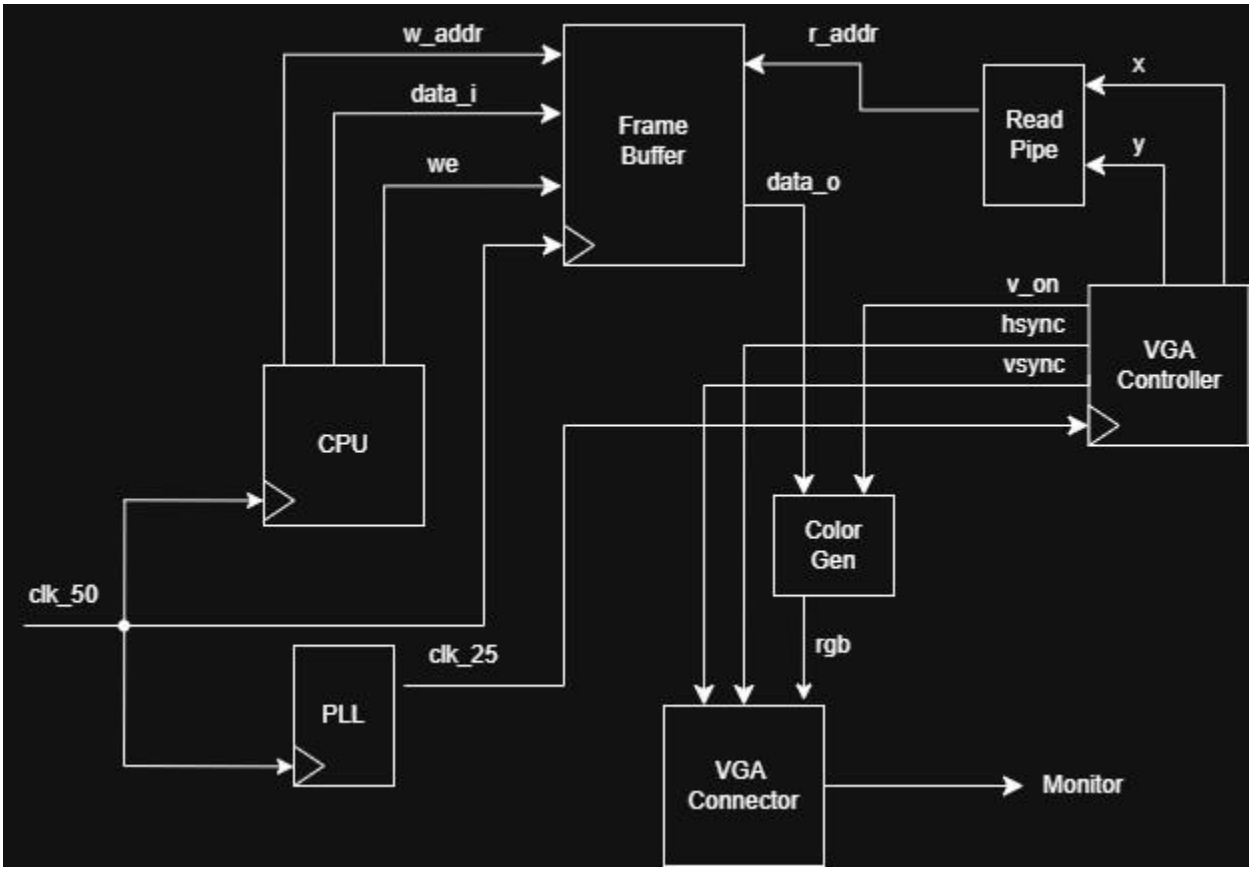

Figure 12: VGA Interface

4.1 Frame Buffer

The Frame Buffer holds the pixel data. It is word-accessible memory in the byte-addressable address range of 0x5000 to 0x17BFF, corresponding to 20kB to 95kB in BRAM. If an instruction in the processor writes to an address in this range, the data ends up in the frame buffer. The write enable (we) signal ensures that only data intended for this range ends up in the buffer. It is important to note that although the frame buffer is written at the clock speed of the CPU (50MHz), data is read at the clock speed of the PLL (25.175MHz) to abide by VGA video timings.

4.2 VGA Controller

The VGA Controller is best described as two counters, x and y. The x counter increments by one pixel every clock cycle. The visible horizontal resolution is 640 pixels, but this becomes 800 pixels when taking the front porch, back porch, and horizontal sync area into account.
From my understanding, these areas are not completely needed and are legacy from VGA originally being meant for old CRT monitors. The buffer area allowed the electron beam to stabilize before snapping back to x = 0 at the end of a row. However, these areas are standard and define the clock speed, which is why they were kept in this design. Once the x counter is within the horizontal sync area, an hsync signal is sent to the VGA connector, which causes the screen to begin writing data on the next row. The y counter follows the same logic, however it only increments by 1 when x reaches the end of the row. The visible vertical resolution is 480

pixels, which becomes 525 pixels when accounting for buffer areas. The VGA Controller outputs a 2D position (x, y), however, the Frame Buffer requires a 1D address to access memory. This is handled by the read pipe.

4.3 Read Pipe

The read pipe converts a 2D address to a 1D address. This is done using Equation 1. This 1D address is fed into the Frame Buffer every clock cycle, causing data to be read and sent to the Color Generator.

$$\begin{aligned} 1D &= 640 * y + x \\ &= 512y + 128y + x \\ &= \ y \ll 9 + y \ll 7 + x \ (Address\ for\ 1\ pixel) \\ &= y \ll 5 + y \ll 3 + x \gg 4 \ (Addess\ for\ 16\ pixels, AKA\ 1\ word) \end{aligned}$$

1

4.4 Color Generator

The Color Generator takes in the 32-bit word read from the Frame Buffer and splits it into individual pixels to be sent to the VGA Connector. This design uses 2bpp, meaning that each pixel can be one of four colors: black, red, blue, or green. Hence, each word read from memory contains 16 pixels. The generator selects one of these pixels, and decodes it into a signal “rgb” so that an input results in an output with exactly one high bit representing one color (2 to 4 decoder). This bit is replicated 4 times due to the DAC, and sent to the connector as an analog color signal.

5.0 Memory Mapped I/O

The project also implements basic Memory Mapped I/O (MMIO). Each input signal is assigned to one address in memory corresponding to a word of data. Each clock cycle, a write and a read occur simultaneously. The data provided by the input pin is written to memory, and the data in memory corresponding to a passed GPIO address is read.

6.0 Pong Implementation

Pong was implemented on the SoC using C. Although the hardware supports two players, the current version of the game is single player. However, the current software can be easily updated for a two-player game. A short bash script was written to compile the .c file into a .hex file using bare-metal RISC-V toolchain commands. This .hex file could be loaded directly into ROM using the $readmemh command in SystemVerilog.

The code began with constructing a border. The border was designed to be 320x240 pixels centered in the screen. By defining a pointer to an address corresponding to the Frame Buffer, and setting the data at that address, the color of the desired pixels could be directly modified. To make the code slightly more efficient, the border was only drawn once and never cleared. This

means that the player and ball should never directly interfere with the memory addresses holding the border. This was taken into account by limiting move space to just inside the boundary.

The player was designed to sit just above the bottom edge, and to be able to move horizontally. By rapidly clearing the current position and drawing pixels at a new position on a user input, the player is able to move. The player movement function was kept in a while(1) loop to constantly execute. However, a delay function (a big for loop) was added to reduce movement speed by taking up clock cycles. This code could likely be made much more efficient by adding interrupts. The player was bounded just inside the left and right edges which ensured that the border pixels were not overwritten.

The ball works the same way as the player. The old spot is cleared, then it's checked for collisions, moved, and redrawn. Ball position is a pointer, and vx/vy control how fast it moves horizontally and vertically each frame.

The paddle check looks at whether the ball is close enough vertically (based on how fast it's moving, so it doesn't skip past the paddle at high speed) and whether it lines up horizontally with the paddle. If both are true, the ball bounces. If it gets near the bottom border without bouncing, that counts as a miss and the ball resets to center. Hitting the side walls or top border is simpler, just flips the direction when the ball reaches those edges.

7.0 Testing Methodology

Frequent testing was key during the design process. Rather than leaving tests for the very end of the project, testbenches were written for individual modules in the design, including the PC, ROM, complete IF/ID stage, and the top-level CPU. To become more proficient in SystemVerilog, I wrote my own testbenches rather than using online test suites. The testbenches primarily consist of PASS/FAIL tests. Modelsim was used to view waveforms for debugging. Figure 13 shows an example waveform.

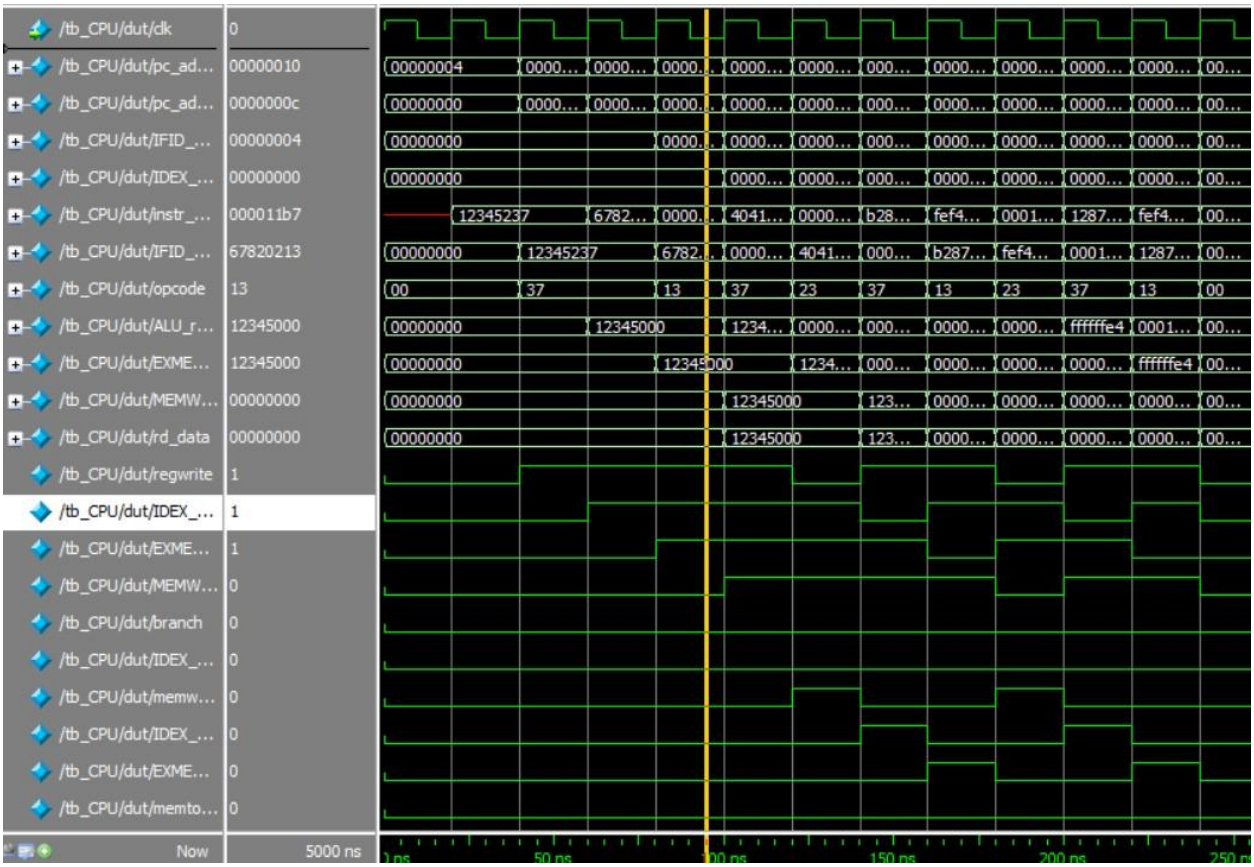

Figure 13: Example waveform during testing

8.0 Results & Performance

This section highlights metrics indicating the performance of the processor, including Fmax, CPI, and the critical path.

8.1 CPI & Fmax

The SoC passed timing at 50MHz, with an Fmax of 51.67MHz, leaving 0.648ns of slack. Through counting how many instructions passed through to WB and disregarding shadow instructions on branches, the CPI (Cycles Per Instruction) was calculated to be 1.343 on a testbench when running Pong. It is important to note that this method of testing is likely not fully reliable. Originally, the datapath was tested with the branch control unit being in the MEM stage. This resulted in a CPI of 1.274. This version would result in an additional branch delay, which would should have a higher CPI than the version where the branch control unit is in EX. Unexpectedly, moving the unit to EX increased CPI to 1.343, rather than decreasing it. Despite this, the final datapath keeps the branch control unit in the EX stage in hope that the unexpected result is error of informal testing. In the future, the Zicsr extension will be implemented to allow for complete testing.

8.2 Critical Path

The critical path was found using Quartus TimeQuest's Report Timing tool under the Slow 1200mV 85C model. As shown in Figure 14, the critical path starts at the rs1 address held in the ID/EX buffer, runs through the forwarding unit to the EX-stage MUXs, and ends at the PC's input select mux. Total data delay on this path is 19.262ns against a 20ns clock period, leaving 0.648ns of slack. The critical path can be seen highlighted red in Figure 15. It makes sense for this to be the critical path, as the signal travels through the most computationally heavy modules (Forwarding Unit, ALU, Branch Control) as well as multiple MUX's, in a single clock cycle.

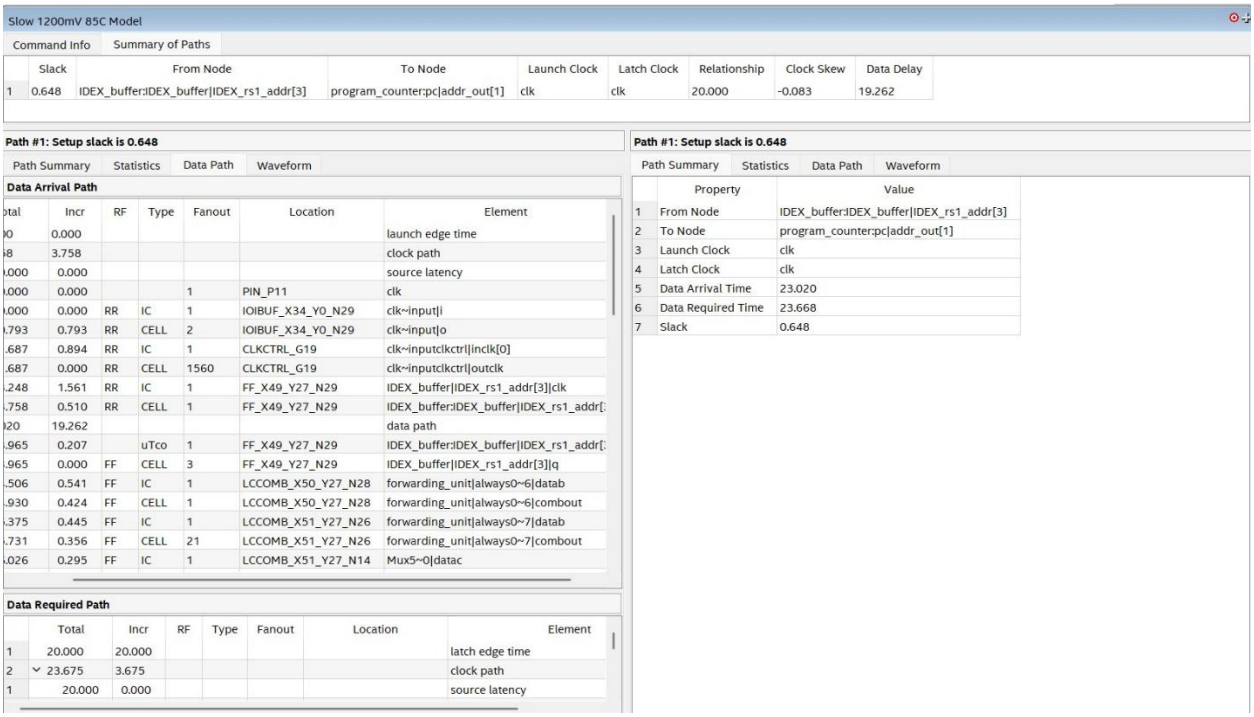

Figure 14: Quartus TimeQuest's Report Timing

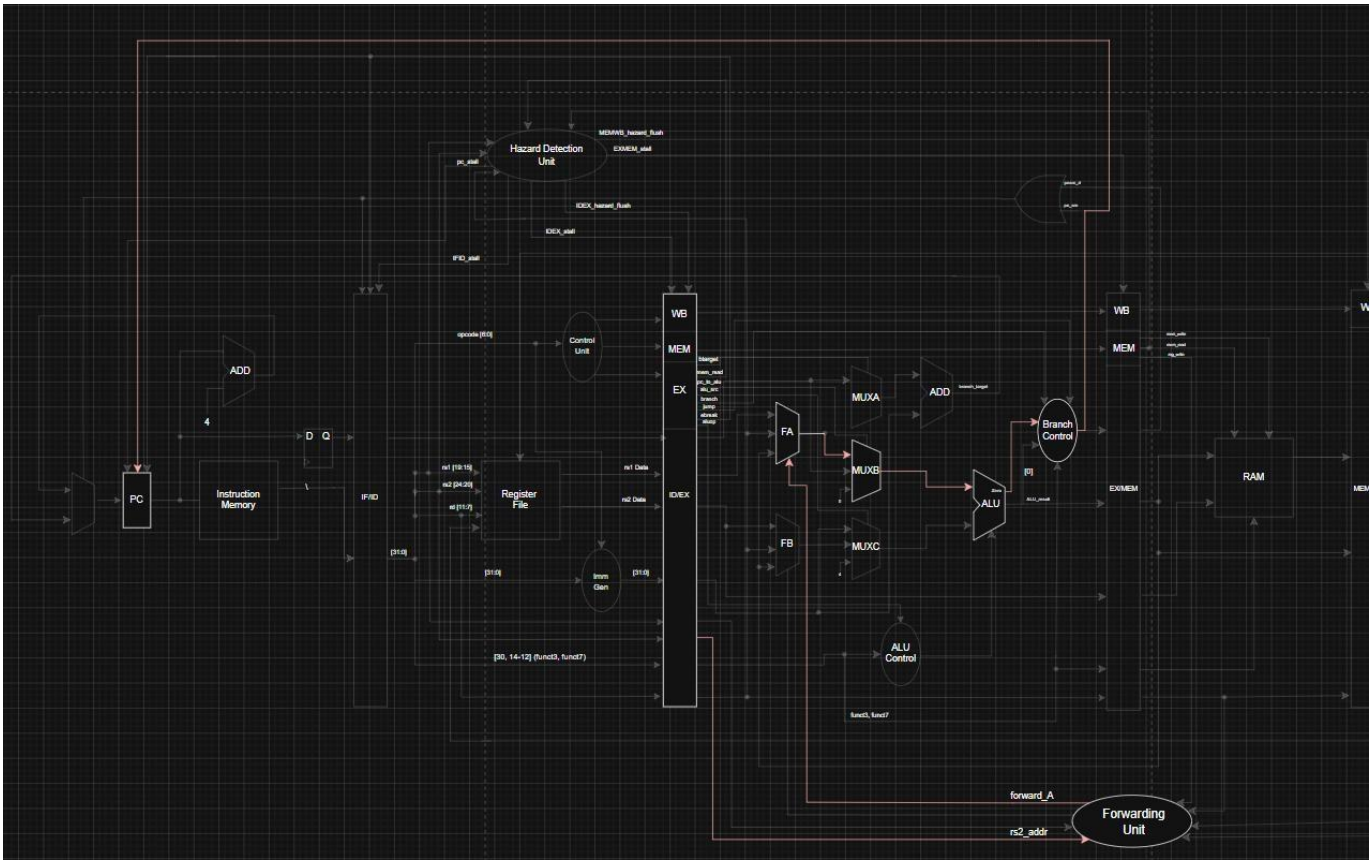

Figure 15: Critical Path

9.0 Final Remarks

This was an extremely fun and rewarding project. I was able to use what I learned in my recent Digital Logic/Computer Architecture courses to build a complete system with minimal abstraction. I learned many different skills, including SystemVerilog, Embedded C, Git, CPU architecture, video graphics, and more. Most importantly, I learned how to self-teach concepts.

Implementing a functional game on the SoC was the most challenging aspect of the project. Although writing the C code was straightforward, to have the game work as expected meant that both the CPU and VGA had to mostly be bug-free. Combined with faulty hardware delays, going from a working simulation to a functioning program on the FPGA took a few extra weeks. But being able to see the whole thing come together through an actual working game made it worth it.

In the future, I hope to continue upgrading the CPU. It could benefit from adding interrupts, branch predictions, a cache, CSR support for formal CPI testing, or support for M/V-type extensions. I also want to explore Out-of-Order/Superscalar processors to see how they may compare to this pipelined version.

References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[2] P. P. Chu, *FPGA Prototyping by SystemVerilog Examples: Xilinx MicroBlaze MCS SoC*. Hoboken, NJ, USA: John Wiley & Sons, Inc., 2018.

[3] C. Hamacher, Z. Vranesic, S. Zaky, and N. Manjikian, Computer Organization and Embedded Systems, 6th ed. New York, NY: McGraw-Hill, 2012.

[4] Terasic Inc., "DE10-Lite User Manual," Oct. 17, 2022. [Online]. Available: https://www.terasic.com.tw/attachment/archive/1081/DE10-Lite_User_manual.pdf

[5] A. Waterman and K. Asanović, Eds., "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," Document Version 20190608-Base-Ratified, SiFive Inc. and Univ. of California, Berkeley, Jun. 8, 2019. [Online]. Available: https://riscv.org/technical/specifications/

[6] "RV32I, RV64I Instructions," *riscv-isa-pages*. [Online]. Available: https://msyksphinzself.github.io/riscv-isadoc/html/rvi.html

Appendix

This appendix shows the detailed CPU datapath which can be viewed more clearly here.

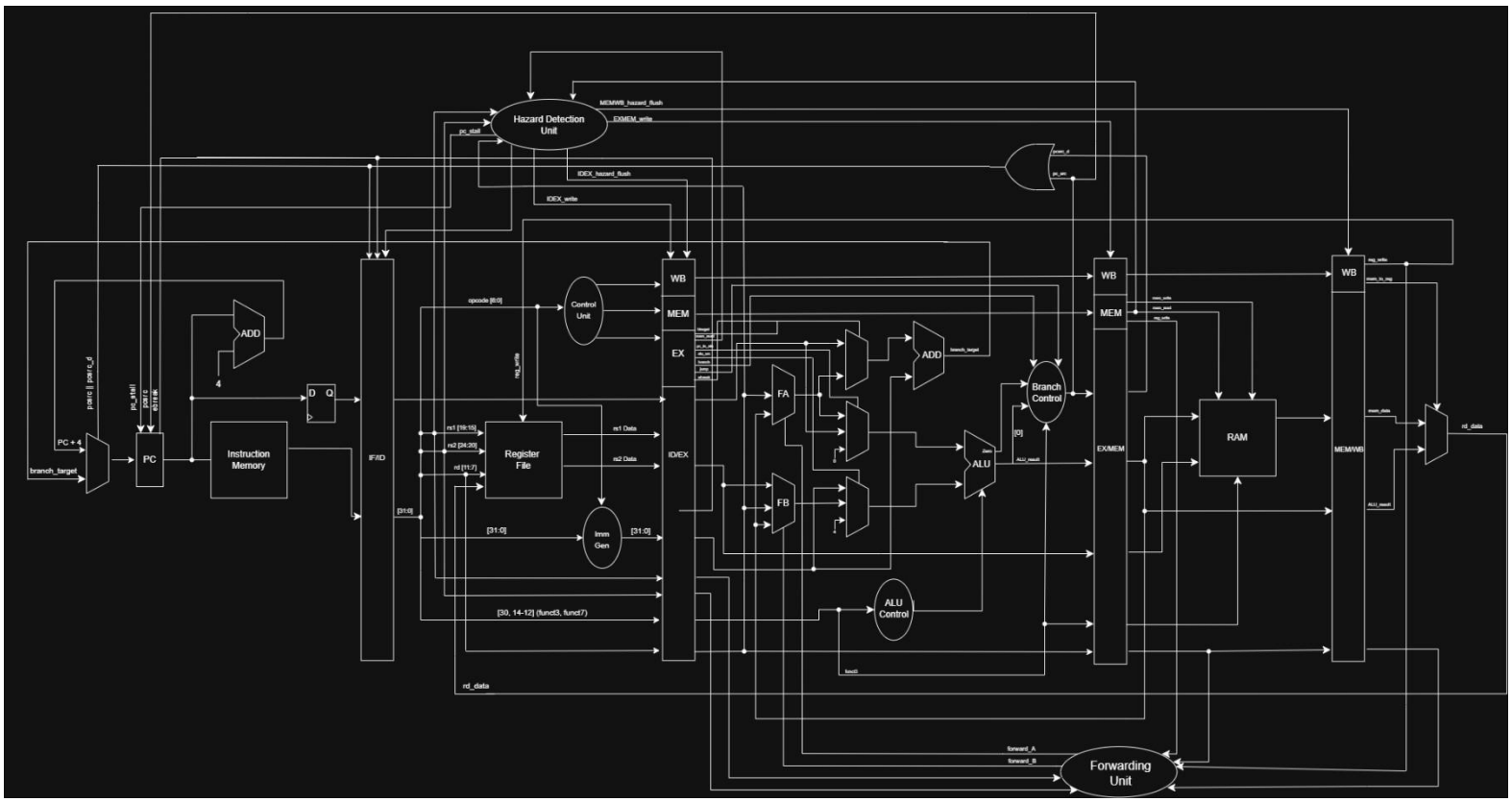

Figure 16: Detailed CPU datapath